



Introduction to Helm

Helm is a tool that automates the creation, packaging, configuration, and deployment of Kubernetes applications by combining your configuration files into a single reusable package

Managing Kubernetes applications across the development lifecycle brings its own set of challenges, including version management, resource allocation, updating, and rollbacks. Helm provides one of the most accessible solutions to this problem, making deployments more consistent, repeatable, and reliable

What is a Helm Chart?

- A Helm chart is a fundamental concept in Helm's ecosystem, representing a packaged set of pre-configured Kubernetes resources. Essentially, it functions as a convenient and portable way to define, install, and manage applications on Kubernetes clusters
- Charts encapsulate all the necessary configuration files, dependencies, and deployment instructions needed for a particular application. This includes YAML configuration files for deployments, services, secrets, and config maps that define the desired state of your application
- They provide a standardized and efficient method for packaging, distributing, and versioning complex applications, contributing to Helm's role as a powerful package manager for Kubernetes
- Additionally, each Helm chart can be versioned and managed independently, making it easy to maintain multiple versions of an application with different configurations

```
my-chart/
|-- Chart.yaml
|-- values.yaml
|-- charts/
|   |-- dependency-chart/
|       |-- charts/
|       |-- templates/
|-- templates/
|   |-- deployment.yaml
|   |-- service.yaml
|   |-- config.yaml
|   |-- secrets.yaml
|   |-- _helpers.tpl
|-- requirements.yaml
|-- NOTES.txt
|-- values.schema.json
|-- _config/
|   |-- additional-config.yaml
```

1. Chart.yaml

- The Chart.yaml file is a metadata file that plays a central role in Helm charts.
- Enables Helm to understand the chart, making it shareable, versioned, and easy to manage.
- This is where other charts on which the helm chart that is being structured depends on are stored.

```
apiVersion: v2
name: my-chart
description: A Helm chart for my application
version: 0.1.0
appVersion: 1.0.0
keywords:
  - helm
  - chart
  - application
```

2. values.yaml

- This is the file in which all the values that are to be injected into the templates are defined. Similar to terraform, Values.yaml is the same as helms variable.tf file.
- The values.yaml file plays a pivotal role in Helm charts, serving as a configuration file that encapsulates default values for the various parameters used within the chart's templates.
- When installing a Helm chart, users have the flexibility to override these default values.

```
# values.yaml
replicaCount: 3
image:
  repository: nginx
  tag: latest
service:
  port: 80
```

3. Charts/

- This is where other charts on which the helm chart that is being structured depends on are stored. There might be a need to call another chart for letting the chart function properly.
- The charts/ directory within a Helm chart serves as a container for subcharts—other charts upon which the primary Helm chart depends.

```
|-- charts/
|   |-- frontend/
|       |-- templates/
|           |-- deployment.yaml
|   |-- backend/
|       |-- templates/
|           |-- deployment.yaml
|-- templates/
|   |-- deployment.yaml
|-- values.yaml
|-- Chart.yaml
```

4. templates/

- The templates/ directory is a core component within Helm charts, housing Kubernetes resource definitions.
- This is the folder where the actual manifest that is being deployed with the chart is put.
- For instance, for deploying an nginx deployment that needs a service, configmap and secrets, there would be a deployment.yaml, service.yaml, config.yaml and secrets.yaml all in the template dir.
- They will all get their values from values.yaml from these.

```
my-chart/  
|-- templates/  
|   |-- deployment.yaml  
|   |-- service.yaml  
|   |-- config.yaml  
|   |-- secrets.yaml  
|-- values.yaml  
|-- Chart.yaml
```

/templates files:-

- deployment.yaml: Contains the configuration for a Kubernetes Deployment, specifying how the application should run and scale.
- service.yaml: Defines a Kubernetes Service, exposing the application to other services within the cluster.
- config.yaml: Represents a ConfigMap, a mechanism to inject configuration data into the application.
- secrets.yaml: Describes Kubernetes Secrets, which are used to store sensitive information securely.

These files in the templates/ directory collectively define the Kubernetes resources necessary for deploying and configuring the application encapsulated by the Helm chart.

4.1 deployment.yaml

- This file defines a Kubernetes Deployment, which is a resource object in Kubernetes that provides declarative updates to applications. A Deployment allows you to describe an application's life cycle, such as which images to use for the app, the number of pod replicas, and the way to update them.

```
# File: mychart/templates/deployment.yaml

apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Release.Name }}-myapp
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
        - name: myapp-container
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
          ports:
            - containerPort: {{ .Values.service.port }}
```

- 'apiVersion' and 'kind' specify the Kubernetes API version and resource kind, respectively.
- 'metadata' contains the name of the Deployment, which is generated based on the Helm release name.
- 'spec' defines the desired state of the Deployment.
- 'replicas' specifies the number of replicas (pods) to run.
- 'selector' is used to match the pods controlled by this Deployment.
- 'template' is the pod template for the Deployment.
- 'metadata' contains labels that match the selector.
- 'containers' defines the container(s) running in the pod, including the image and ports.

4.2 service.yaml

This file defines a Kubernetes Service, which enables external access to the application and routes traffic to the pods managed by a Deployment.

```
# File: mychart/templates/service.yaml

apiVersion: v1
kind: Service
metadata:
  name: {{ .Release.Name }}-myapp-service
spec:
  selector:
    app: myapp
  ports:
    - protocol: TCP
      port: {{ .Values.service.port }}
      targetPort: 80
  type: {{ .Values.service.type }}
```

- 'apiVersion' and 'kind' specify the Kubernetes API version and resource kind, respectively.
- 'metadata' contains the name of the Service, which is generated based on the Helm release name.
- 'spec' defines the desired state of the Service.
- 'selector' is used to match the pods controlled by the corresponding Deployment.
- 'ports' specifies the ports on which the Service will listen.
- 'type' indicates the type of Service, such as ClusterIP, NodePort, or LoadBalancer. The type is sourced from the Helm values file.

4.3 config.yaml

- The config.yaml file is often used to manage configuration settings for an application. It allows you to separate configuration from code, making it easier to customize the behavior of your application without modifying the application itself.

```
# File: mychart/templates/config.yaml

# Application Configuration
database:
  host: {{ .Values.config.database.host | default "localhost" }}
  port: {{ .Values.config.database.port | default 5432 }}
  username: {{ .Values.config.database.username | default "myuser" }}
  password: {{ .Values.config.database.password | default "mypassword" }}
```

- '.Values.config.database.host', '.Values.config.database.port', '.Values.config.database.username', and '.Values.config.database.password' are placeholders for configuration values.
- The '|' default filter is used to provide default values if the corresponding values are not specified in the Helm values file.

4.4 secrets.yaml

- The 'secrets.yaml' file is typically used to manage sensitive information securely. Secrets can be mounted as files or used as environment variables within your application, and Helm helps manage these secrets during deployment.

```
# File: mychart/templates/secrets.yaml

apiVersion: v1
kind: Secret
metadata:
  name: {{ .Release.Name }}-myapp-secrets
type: Opaque
data:
  username: {{ .Files.Get "secrets/username.txt" | b64enc | quote }}
  password: {{ .Files.Get "secrets/password.txt" | b64enc | quote }}
```

- '.Release.Name' is used to generate a unique name for the secret based on the Helm release name.
- 'type: Opaque' indicates a generic secret type.
- 'data' contains the actual secret information. Here, it loads the contents of two files ('username.txt' and 'password.txt') from the 'secrets/' directory, encodes them in base64, and includes them in the secret.

5. Other helm chart files

1. `_helpers.tpl`: A Helm template file that contains reusable template snippets and functions. These can be shared across multiple charts for consistency.
2. `requirements.yaml`: Specifies dependencies for the Helm chart, allowing you to declare other charts that must be present for the primary chart to function properly.
3. `NOTES.txt`: A file containing post-installation and post-upgrade notes that are displayed to the user after the Helm chart is deployed.
4. `LICENSE`: This file includes the license information for the Helm chart.
5. `README.md`: This file provides documentation and information about the Helm chart.
6. `.helmignore`: Similar to `.gitignore`, this file specifies files or directories that should be ignored when packaging the chart.
7. `tests/`: This directory may contain test files and scripts to validate the Helm chart.

5.1 _helpers.tpl

- The `_helpers.tpl` file in a Helm chart is a template helper file that contains reusable template snippets or functions. It allows you to define reusable pieces of code that can be included in other template files in your Helm chart.
- The filename typically starts with an underscore (`_`) to indicate that it's a helper file and not meant to be rendered as a standalone Kubernetes resource.

```
# File: mychart/templates/_helpers.tpl

{{/*
Create a helper function to generate labels for resources.
Usage: {{ include "mychart.labels" $labels | indent 4 }}
*/}}
{{- define "mychart.labels" -}}
    {{- $labels := .Values.labels | merge .Chart.labels | merge $ -}}
    {{- with .Values.podLabels -}}
        {{- $labels = $labels | merge . -}}
    {{- end -}}
    labels:
        {{- range $key, $value := $labels }}
            {{ $key }}: {{ quote $value }}
        {{- end }}
    {{- end }}

```

5.2 requirements.yaml

- The 'requirements.yaml' file is used to declare dependencies for your Helm chart. It specifies other charts that your chart depends on, including the chart name, version constraints, and any additional settings.

```
# File: mychart/requirements.yaml

dependencies:
  - name: nginx
    version: "1.2.3"
    repository: "https://charts.example.com/stable"

```

- dependencies is a list of dependencies that your chart relies on.
- name is the name of the dependency chart.
- version is the version constraint for the dependency. Helm will attempt to use a version that satisfies this constraint.
- repository is the URL of the Helm chart repository where the dependency can be found.

5.3 NOTES.txt

- The NOTES.txt file in a Helm chart is used to provide post-installation notes or information that is displayed to the user after a Helm chart is deployed. These notes can include instructions, tips, or any other relevant information that users should be aware of after the deployment process.
- When users install your Helm chart, Helm will automatically display the content of the NOTES.txt file in the command line. Users can follow the provided instructions and notes to access and interact with the deployed application.

Thank you for using MyChart!

To access your application, use the following commands:

1. Get the application URL by running:

```
export SERVICE_IP=$(kubectl get svc --namespace  
default mychart-myapp-service -o jsonpath='{.status.loadBalancer.ingress[0].ip}')  
echo "Application URL: http://$SERVICE_IP:{{.Values.service.port }}"
```

2. Open your browser and navigate to the provided URL.

3. If you encounter any issues, check the application logs:

```
kubectl logs --namespace default -l app=myapp
```

Happy deploying!

5.4 LICENSE

- In a Helm chart, the LICENSE file typically contains information about the licensing terms and conditions for the Helm chart. It provides users and developers with details about how they can use, modify, and distribute the chart.

MIT License

Copyright (c) [year] [author]

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

- MIT License: Specifies the type of license (in this case, the MIT License).
- Copyright: States the copyright holder or holders.
- Permission is hereby granted...: Outlines the permissions granted to users under the license.
- The above copyright notice and this permission notice...: Specifies that the license terms must be included in all copies or substantial portions of the software.
- THE SOFTWARE IS PROVIDED "AS IS"...: Includes a disclaimer of warranty.
- IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS...: Includes a limitation of liability.

5.5 README.md

- The README.md file in a Helm chart serves as documentation to provide users with information about the chart, its purpose, how to use it, and any other relevant details.
- Replace my-release with the desired release name.

```
# MyChart

MyChart is a Helm chart for deploying and managing MyApplication on Kubernetes.

## Installation

To install MyChart, use the following Helm command:

```bash
helm install my-release ./mychart
```

## 5.6 .helmignore

- It specifies patterns of files and directories that Helm should ignore when packaging the chart. Helm uses the '.helmignore' file to determine which files to exclude when creating a chart package (tar.gz file) using the helm package command.

```
Ignore files and directories generated during development or build
node_modules/
dist/
build/

Ignore editor-specific files
.vscode/
.idea/

Ignore temporary files
*.swp
*~
```

- node\_modules/, dist/, and build/ are directories that might be generated during development or build processes and are typically not needed in the packaged Helm chart.
- .vscode/ and .idea/ are directories specific to Visual Studio Code and IntelliJ IDEA, respectively, and are not necessary in the packaged Helm chart.
- \*.swp and \*~ are examples of temporary files created by certain editors and are generally not needed in the packaged Helm chart.

## 5.7 tests/

- In a Helm chart, the tests/ directory is commonly used to store test files and scripts that can be run to verify the correctness and functionality of the Helm chart. These tests are often automated and can be executed as part of a continuous integration (CI) pipeline or manually by users.

## Cheat Sheet

Helm cheatsheet featuring all the necessary commands required to manage an application through Helm

### Basic interpretations/context

#### Chart:

- It is the name of your chart in case it has been pulled and untarred.
- It is <repo\_name>/<chart\_name> in case the repository has been added but chart not pulled.
- It is the URL/Absolute path to the chart.

#### Name:

- It is the name you want to give to your current helm chart installation.

#### Release:

- Is the name you assigned to an installation instance.

#### Revision:

- Is the value from the Helm history command

#### Repo-name:

- The name of a repository.

#### DIR:

- Directory name/path

## Chart Management

helm create <name>	Creates a chart directory along with the common files and directories used in a chart.
helm package <chart-path>	Packages a chart into a versioned chart archive file.
helm lint <chart>	Run tests to examine a chart and identify possible issues
helm show all <chart>	Inspect a chart and list its contents
helm show values <chart>	Displays the contents of the values.yaml file
helm pull <chart>	Download/pull chart
helm pull <chart> --untar=true	If set to true, will untar the chart after downloading it
helm pull <chart> --verify	Verify the package before using it
helm pull <chart> --version <number>	Default-latest is used, specify a version constraint for the chart version to use
helm dependency list <chart>	Display a list of a chart's dependencies

## Install and Uninstall Apps

helm install <name> <chart>	Install the chart with a name
helm install <name> <chart> --namespace <namespace>	Install the chart in a specific namespace
helm install <name> <chart> --set key1=val1,key2=val2	Set values on the command line (can specify multiple or separate values with commas)
helm install <name> <chart> --values <yaml-file/url>	Install the chart with your specified values
helm install <name> <chart> --dry-run --debug	Run a test installation to validate chart (p)
helm install <name> <chart> --verify	Verify the package before using it
helm install <name> <chart> --dependency-update	update dependencies if they are missing before installing the chart
helm uninstall <name>	Uninstall a release

## Perform App Upgrade and Rollback

helm upgrade <release> <chart>	Upgrade a release
helm upgrade <release> <chart> --atomic	If set, upgrade process rolls back changes made in case of failed upgrade
helm upgrade <release> <chart> --dependency-update	update dependencies if they are missing before installing the chart
helm upgrade <release> <chart> --version <version_number>	specify a version constraint for the chart version to use
helm upgrade <release> <chart> --values	specify values in a YAML file or a URL (can specify multiple)
helm upgrade <release> <chart> --set key1=val1,key2=val2	Set values on the command line (can specify multiple or separate values)
helm upgrade <release> <chart> --force	Force resource updates through a replacement strategy
helm rollback <release> <revision>	Roll back a release to a specific revision
helm rollback <release> <revision> --cleanup-on-fail	Allow deletion of new resources created in this rollback when rollback fails

## List, Add, Remove, and Update Repositories

helm repo add <repo-name> <url>	Add a repository from the internet
helm repo list	List added chart repositories
helm repo update	Update information of available charts locally from chart repositories
helm repo remove <repo_name>	Remove one or more chart repositories
helm search repo <keyword>	Search repositories for a keyword in charts
helm search hub <keyword>	Search for charts in the Artifact Hub or your own hub instance

## Helm Release monitoring

helm list	Lists all of the releases for a specified namespace
helm list --all	Show all releases without any filter applied, can use --a
helm list --all-namespaces	List releases across all namespaces
helm list --date	Sort by release date
helm list --deployed	Show deployed releases
helm list --pending	Show pending releases
helm list --failed	Show failed releases
helm list --uninstalled	Show uninstalled releases
helm list --superseded	Show superseded releases
helm list -o yaml	Prints the output in the specified format
helm status <release>	This command shows the status of a named release
helm status <release> --revision <number>	if set, display the status of the named release with revision
helm history <release>	Historical revisions for a given release
helm env	Env prints out all the environment information in use by Helm

## Download Release Information

helm get all <release>	A human readable collection of information about the notes, hooks, supplied values, and generated manifest file of the given release
helm get hooks <release>	This command downloads hooks for a given release. Hooks are formatted in YAML and separated by the YAML
helm get manifest <release>	A manifest is a YAML-encoded representation of the Kubernetes resources that were generated from this release's chart(s)
helm get notes <release>	Shows notes provided by the chart of a named release
helm get values <release>	Downloads a values file for a given release

## Plugin Management

helm plugin install <path/url1>	Install plugins
helm plugin list	View a list of all installed plugins
helm plugin update <plugin>	Update plugins
helm plugin uninstall <plugin>	Uninstall a plugin